茗花智能汇报系统 - 软件设计文档

1. 系统概述

1.1 目标

将茗花改造成一个可自我进化的智能汇报 **Web** 应用，通过聊天窗口接收用户指令，自动执行数据分析并展示结果，同时记录对话内容，由 AI 自动分析并生成产品设计，实现功能迭代。

1.2 核心特性

- 🗣️ 聊天式交互: 自然语言输入，动态展示结果
- 🤖 **AI** 外挂: 集成 OpenClaw 作为智能大脑
- 🔄 自我进化: 记录对话 → AI 分析 → 自动改代码
- 🛡️ 安全分级: 目录级代码修改权限控制

1.3 设计原则

- 最小侵入: 尽可能少的改动原项目文件目录
- 通用可移植: 进化系统作为独立模块，可复用到其他应用
- 解耦设计: 聊天 UI、意图识别、进化引擎均可独立使用

---

2. 系统架构

```
┌──────────────────────────────────────────────────────────────────┐
│                          用户层 (Web UI)                          │
│  ┌──────────────────────────────────────────────────────────┐    │
│  │                          聊天窗口                          │    │
│  │  用户输入 → 意图识别 → 代码执行 → 结果展示                 │    │
│  └──────────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────┐
│                        业务逻辑层 (Backend)                       │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────┐        │
│  │ 意图识别器   │  │ 代码执行器   │  │   对话记录器     │        │
│  │ Intent       │  │ Executor     │  │ ConversationLog  │        │
│  └──────────────┘  └──────────────┘  └──────────────────┘        │
│                                                                   │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────┐        │
│  │ 结果渲染器   │  │ 权限管理器   │  │   进化引擎       │        │
│  │ Renderer     │  │ Permission   │  │ EvolutionEngine  │        │
│  └──────────────┘  └──────────────┘  └──────────────────┘        │
└──────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────┐
│                          AI 层 (OpenClaw)                         │
│  ┌──────────────────────────────────────────────────────────┐    │
│  │  意图理解 / 代码生成 / 产品设计 / 代码修改                 │    │
│  │                                                          │    │
```

```
└──────────────────────────────────────────────────────────────────────┘
```

---

3. 模块设计

3.1 意图识别器 (IntentClassifier)

功能: 解析用户输入，判断用户意图

支持意图:

- VIEW_WEEKLY_REPORT: 查看周报
- VIEW_MONTHLY_REPORT: 查看月度报告
- VIEW_CROSS_WEEK: 查看跨周对比
- QUERY_STORES: 查询店铺数据
- QUERY_FRUITS: 查询水果数据
- CUSTOM: 自定义需求

3.2 代码执行器 (CodeExecutor)

功能: 根据意图执行对应的代码逻辑，对接原项目的报表生成模块

核心方法:

```
def execute(intent_type: str, params: Dict) -> ExecutionResult:
    """执行意图对应的代码"""
```

3.3 对话记录器 (ConversationLogger)

功能: 记录所有用户对话，支持后续 AI 分析

存储: JSON 文件（预留数据库接口）

3.4 权限管理器 (PermissionManager)

功能: 管理代码修改的权限级别

权限级别:

- FORBIDDEN: 禁止修改
- CONFIRM_REQUIRED: 需用户确认
- AUTO: 自动修改

3.5 进化引擎 (EvolutionEngine)

功能: 分析对话，生成设计文档，自动修改代码，版本管理

触发方式:

- 手动触发：用户在界面点击"开始进化"按钮
- 定时触发：用户配置 Cron 表达式

版本管理:

```
evolution_history/
├── v1.0_2026-03-16/
│   ├── design.md       # 设计文档
│   ├── changelog.md    # 变更日志
│   └── README.md      # 版本说明
└── ...
```

---

4. 目录结构

```
茗花by_claw/
├── minghua_evo/                # 自我进化系统（独立模块）
│   ├── __init__.py
│   ├── core/
│   │   ├── intent_classifier.py    # 意图识别器
│   │   ├── code_executor.py      # 代码执行器
│   │   └── conversation_logger.py # 对话记录器
│   ├── services/
│   │   └── evolution_engine.py # 进化引擎
│   ├── api/
│   │   └── routes.py           # FastAPI 路由
│   ├── config/
│   │   ├── permissions.yaml    # 权限配置
│   │   └── settings.yaml      # 系统设置
│   ├── design/                 # AI 设计文档
│   └── evolution_history/      # 版本历史
│
├── src/                        # 原项目源代码（最小改动）
│   ├── core/
│   ├── features/
│   ├── data_loader.py
│   ├── generators.py
│   └── config.py
│
├── frontend/                    # Vue 3 前端
│   └── dist/
│
├── data/                       # 数据目录
│   ├── conversations/          # 对话记录
│   └── 3月/
│
├── server.py                    # Web 服务器入口
├── fruits.json                 # 水果分类配置
└── stores.json                 # 店铺类型配置
```

---

5. API 设计

5.1 聊天 API

POST /api/chat

```
// Request
{ "message": "查看本月运营情况" }

// Response
{
  "intent": "VIEW_MONTHLY_REPORT",
  "reply": "为您生成3月月度汇总报告...",
  "data": { ... },
  "render_type": "chart"
}
```

5.2 进化引擎 API

- `POST /api/evolve/trigger` - 手动触发进化
- `GET /api/evolve/status` - 获取进化状态
- `GET /api/evolve/versions` - 获取版本历史
- `POST /api/evolve/schedule` - 设置定时任务

---

6. 安全策略

6.1 权限配置 (permissions.yaml)

```
permissions:
  src/core/:
    level: "forbidden"
  src/features/:
    level: "confirm"
  src/experiments/:
    level: "auto"
```

6.2 进化引擎安全

- 每次自动修改前备份
- 记录所有代码变更日志
- 用户可回滚任意版本

---

7. 进化功能扩展需求

7.1 意图识别优化

- 【重要】接入真正 AI 意图识别：目前的意图识别是基于规则的正则匹配，需要接入 OpenClaw 实现真正的 AI 意图识别
  - 用户输入自然语言，AI 理解真实意图
  - 支持更复杂的表达方式
  - 支持多轮对话上下文理解
- 优先级关注未识别文本

- 优先级关注未识别文本：对于无法识别的用户输入（CUSTOM 类型），优先记录并纳入进化分析
- 智能过滤闲聊：对于闲聊类内容（如"你好"、"谢谢"、"辛苦了"等与业务无关的寒暄），不纳入功能设计参考
- 意图分类：区分业务需求与闲聊，设置不同的处理策略

7.2 代码修改权限优化

- 白名单机制：设置允许 AI 修改的目录白名单，白名单内的代码可被 AI 自动修改
- 黑名单机制：关键核心文件（如配置文件、数据库连接等）加入黑名单，禁止任何 AI 修改
- 分级管理：
  - 🔴 黑名单（禁止）：config.py、stores.json、fruits.json、数据库相关
  - 🟡 白名单（自动）：src/experiments/、src/plugins/
  - 🟢 待确认（默认）：其他业务代码

7.3 定时进化优化

- 定时任务独立运行：开启定时进化后，不依赖用户输入日志，按时自动执行
- 进化内容来源：
  - 近期用户对话（排除闲聊）
  - 已确认但未应用的设计文档
- 状态记录：记录每次定时进化的执行时间、修改内容、结果

7.4 闲聊过滤机制

- 闲聊识别规则：
  - 问候类："你好"、"早上好"、"晚安"
  - 感谢类："谢谢"、"辛苦了"
  - 肯定类："很好"、"不错"
  - 闲聊类："今天天气"
- 过滤策略：上述类型不加入进化需求池，但仍记录在对话历史中

---

8. 实施计划

- ☒ Phase 1: 基础聊天功能
- ☒ Phase 2: 对话记录与权限
- ☒ Phase 3: 进化引擎
- ☐ Phase 4: 集成 OpenClaw AI
- ☐ Phase 5: 意图识别优化（优先处理未识别文本）
- ☐ Phase 6: 代码权限优化（白名单/黑名单机制）

- ☐ Phase 7: 定时进化优化（独立运行+闲聊过滤）

---

9. 技术选型

项目	选择
Web 框架	FastAPI
前端框架	Vue 3
对话存储	JSON（预留数据库适配接口）
进化触发	手动按钮 + 定时任务（用户可配置）
模块设计	独立包 (minghua_evo)，便于移植

文档版本*: v1.0* 创建日期*: 2026-03-16* 最后更新*: 2026-03-17*